

Virtual Reality Implementation of Steering Hands Motion in Driving Simulator with Unity Technology

Norikazu Ikoma

Dept. of Computer and Information Eng.,
Faculty of Engineering,
Nippon Institute of Technology
Saitama 345-8501, JAPAN

Yuusuke Doi

Dept. of Integrated System Eng.,
School of Engineering,
Kyushu Institute of Technology
Fukuoka 804-8550, JAPAN

Sozo Inoue

Dept. of Human Intelligence Systems,
Graduate School of Life Sci. and Sys. Eng.,
Kyushu Institute of Technology
Fukuoka 808-0196, JAPAN

Abstract—Motivated to achieve visual feedback of body motion in head mounted virtual reality, proposal of this paper is one practical implementation method of a system that captures hands motion of car driver in steering operation via vision sensor and displays the driver's hands over virtual reality graphics governed by Unity technology. It utilizes TCP/IP socket communication with client/server framework for management of passing data among multiple computer as clients. Actual implementation result has been reported under driving simulator facility equipped with two-axes motion actuators and HTC Vive head mounted display for virtual reality based on Unity technology as graphics and driving simulator software engine.

I. INTRODUCTION

Hands motion of car driver in operating steering of an automotive is important information for computer based systems to achieve advanced safety driving support, self-driving in lower-level, and higher-level self-driving with handover between automated system and human driver [1].

There are series of conventional researches [2] [3] [4] [5] [6] [7] to estimate hands motion in steering operation from captured video by a camera set at above the seatbelt roller or depth image sensor set above car driver's head. It can be said that this approach is one promising methodology to obtain such important information of hands motion from vision based sensors including depth image sensor (Kinect) with the aid of GPGPU technology for high performance computing.

To analyze human motion in automotive driving, realistic driving simulators that utilizes virtual reality technologies including head mounted display are efficient due to their high quality of reality provided to test subjects to perform realistic experiments with driving a car.

Significant problems arise in use of such virtual reality systems with head mounted display. That is, how to capture body posture of test subject in motion, including hands motion, and how to achieve visual feedback of body motion via head mounted display.

To overcome the above problems, this paper proposes one implementation method of a system that captures hands motion of car driver in steering operation via vision sensor and displays the driver's hands over virtual reality graphics governed by Game engine Unity. In addition, TCP/IP socket communication with client/server framework supports management of passing data among multiple computer as clients.

Actual implementation result has been reported under driving simulator facility that equips two-axes motion actuators, HTC Vive head mounted display for virtual reality based on Unity technology, and driving simulator software engine. We have made discussion on the proposed method in comparison with other sensor-based technologies such as hand gesture sensor and depth image sensor.

II. BLACKBOARD SERVER SYSTEM

As data flow management for proposed system consisting of several nodes such as sensor node and virtual reality node, a framework of network system with blackboard server and client nodes for data access has been employed. It allows asynchronous data transfer among clients through the server. In the followings subsections, communication protocol between blackboard server and data access client will be described, followed by each description of blackboard server and data access client.

A. Protocol

Protocol of communication between server and client consists of two major categories, management commands and data handling commands. Details of these two categories and commands of each have been described in below.

Management commands are the following three commands, "Register New data" that requests new data registration to the server, "Delete the Registered data" that requests to delete the registered data of the server, and "Disconnect Communication" that disconnects the socket communication channel to the server. Sequence Diagrams of these commands are shown in Fig.1 depicting message passing between client and server.

Data handling commands are the following three commands, "Set Value of the Data" that sends data value of specified data to the server, "Get Value of the Data" that receives data value of specified data from the server, and "Query Size of the Data" that makes query of data registration with its memory size. Sequence Diagrams of these commands are shown in Fig.2 in the same manner of Fig.1.

Each registration of data has been identified by a character (i.e., 1 byte symbol), which has been shown as "ID" in Fig.1 and Fig.2. Other symbols for command and message are summarized in Table I.

TABLE I
SYMBOLS IN PROTOCOL FOR BLACKBOARD SERVER SYSTEM.

Category	Content	Symbol
Command	Register New Data	REG
	Delete the Data	DEL
	Query Size of the Data	QRY
	Set Value of the Data	SET
	Get Value of the Data	GET
Message	Acknowledge with OK	ACK
	NG acknowledgement	NEG

B. Blackboard Server

Blackboard server provides data storage and access to it via TCP/IP communication along with the protocol explained in previous subsection. Multi-thread programming has been employed to provide simultaneous services to multiple clients with the aid of shared memory structure and semaphore to avoid conflict of same memory access by multiple clients.

The blackboard server accepts connection from client at any time and creates new thread after having new connection. After establish the connection, it will work until the client send "Disconnect Communication" command. When "Disconnect Communication" command has been received, the blackboard server will close connection and free all the memory used for the connection. During the connection to a client, the blackboard server accepts the commands shown in Table I from the client. Details of each command are described below.

"Register New Data" command with "ID" from the client makes the server to create new data storage identified by the "ID" with its memory size according to the "size" sent from the client. The server replies to the client whether the request completed or not by sending message "ACK" or "NEG".

"Delete the Data" command with "ID" from the client makes the server to delete the data storage identified by the "ID" followed by message sending of "ACK" or "NEG" whether or not the delete process being completed.

"Set Value of the Data" command with "ID" from the client makes the server to check whether the data storage with "ID" exists or not followed by replying "ACK" or "NEG" to the client. When the data storage exists, the server will send the data stream to the client.

"Get Value of the Data" command with "ID" from the client makes the server to check whether the data storage with "ID" exists or not followed by replying "ACK" or "NEG" to the client. When the data storage exists, the server will receive the data stream from the client.

"Query Size of the Data" has two purposes, 1) checking the existence of the data storage without issuing command "Set Value of the Data" or "Get Value of the Data", and 2) asking size of data storage, specified by the "ID". The server replies with "NEG" when it doesn't exists, while it replies with "ACK" followed by sending the size to the client.

Note that each client can manage multiple data with different "ID"s, where same "ID" provides the same data storage for all clients. This provides data passing mechanism among different clients through the blackboard server.

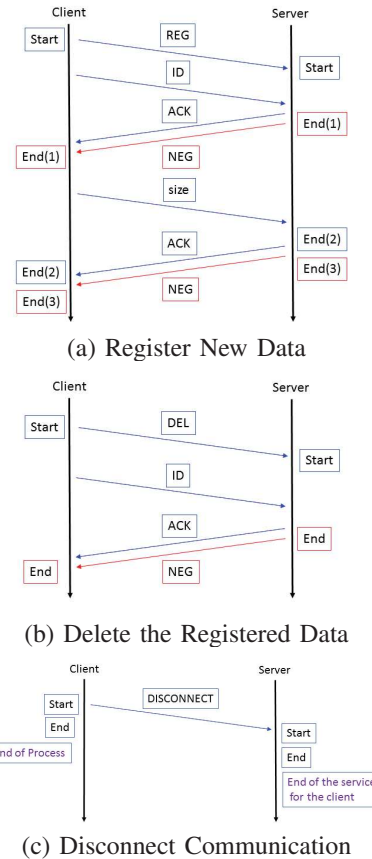


Fig. 1. Sequence Diagram of Blackboard Server System for Management Commands.

C. Data Access Client

Asynchronous data transfer among clients has been achieved by sending data from source client to blackboard server followed by the action of getting data from blackboard server by destination client.

Client for data access firstly create new TCP/IP connection to the blackboard server by specifying IP address and designated port number. After establishing the connection, the client can send commands shown in Table I to the server. Details of each command are briefly described below in the order of typical usage.

Send "Register New Data" command with "ID" to the server when the client is the first one to use the data storage. "Query Size of the Data" can be sent to the server, if necessary, in order to ask registration and/or size of the data storage.

Data access can be made by issuing "Set Value of the Data" command or "Get Value of the Data" command to the server with "ID" for the required data storage. The latest value of the data storage will be processed by these commands.

Send "Delete the Data" command with specific "ID" when the data storage become unnecessary for all the clients.

To terminated TCP/IP connection, issue "Disconnect Communication" command to the server. Then, the connection of the client to the server will be terminated.

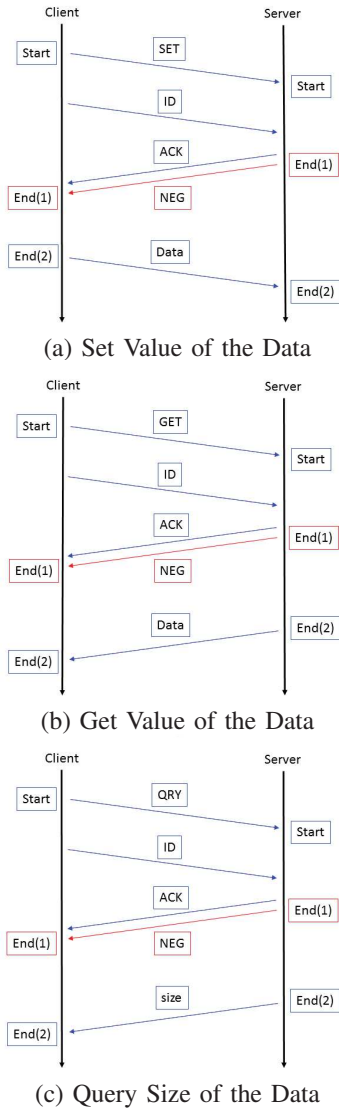


Fig. 2. Sequence Diagram of Blackboard Server System for Data Handling Commands.

III. SENSOR CLIENT FOR HANDS MOTION

Hands motion estimation of a car driver in steering operation from captured video image of USB camera set above seatbelt roller [3] has been employed as a method to capture the hands motion. Output of the method at each time of image frame consists of four angles, grasping angles over steering for left and right hand, denoted by θ_k^L and θ_k^R respectively, and arm direction angles for left and right arms, denoted by φ_k^L and φ_k^R respectively. These angles are concisely represented by a state vector

$$\mathbf{x}_k = [\theta_k^L, \theta_k^R, \varphi_k^L, \varphi_k^R], \quad (1)$$

where integer k denotes discrete time index of image frame.

Through state estimation with particle filter, posterior distribution of the state vector given the series of image frames up to current time, which is denoted by $\mathbf{I}_{1:k}$, has been obtained.

Then, from the posterior distribution $p(\mathbf{x}_k | \mathbf{I}_{1:k})$, representative value $\bar{\mathbf{x}}_k$ such as mean, MAP, has been calculated and sent to the blackboard server within the framework of data access client explained in subsection II-C. We call this client as "Sensor Client for Hands Motion".

In addition with the representative value $\bar{\mathbf{x}}_k$, discrete time index k and existence flags of left and right hands, denoted by f_L and f_R respectively, are sent to the blackboard server. Where the existence flag takes value 1 when the corresponding hand exists in the scene with a context of steering operation, otherwise it takes value 0.

IV. VIRTUAL REALITY CLIENT

The sent data of hands motion of a car driver in the form of representative value $\bar{\mathbf{x}}_k$, along with discrete time index k and existence flags of the hands f_L and f_R , stored in the blackboard server are accessed by a data access client within the framework explained in subsection II-C. Received data $\bar{\mathbf{x}}_k$ stored in the client are displayed via computer graphics including virtual reality facility. We call this client as "Virtual Reality Client".

Each time this client receives new value of discrete time index k , in comparison with previously received value stored in the client's memory, it re-draws the computer graphics content according to the new of representative value $\bar{\mathbf{x}}_k$ under the condition that existence flag of each hand takes value 1.

Communication interval of this client is governed by a kind of timer interruption in which the user can specify the value of time interval according to the expected rate of data update by "Sensor Client for Hands Motion".

Driving simulator function is also provided by this client in a framework of game engine such as "Unity". It generates views of head mounted display worned by a human driver and motion command to the actuators of the driving simulator facility.

V. IMPLEMENTATION

Driving simulator facility used for the implementation, being shown in Fig.3, has two-axes motion actuators and HTC Vive head mounted display for virtual reality based on Unity technology as graphics and driving simulator software engine based on PC with Windows 10 operating system by Microsoft corporation. Note on Fig.3 that actual usage of the facility is wearing the head mounted display and seatbelt.

The proposed mechanism with blackboard server and data access clients explained in section II has been implemented in the driving simulator facility with its software environment using Visual Studio 2015 C/C++ by Microsoft corporation and game engine Unity provided by Unity technologies corporation with Script Language C# as so-called "assets".

As a first step for the implementation, by excluding the virtual reality part and driving simulator part from the "Virtual Reality Client", thus, only "Sensor Client for Hands Motion", blackboard server, and data access client with game engine Unity with C# scripts for single screen display have been implemented as shown in Fig.4.



(a) Cabin Part : Seat, Steering, and Pedals.



(b) Side-Front View.



(c) Back View.



(d) Side View with Driver.

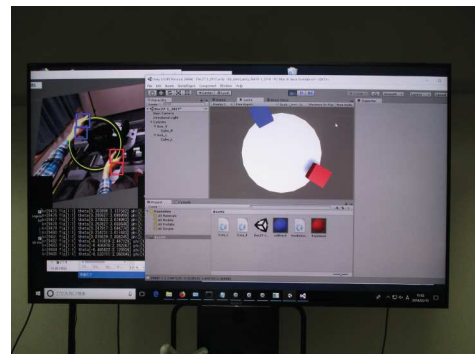
Fig. 3. Driving Simulator.



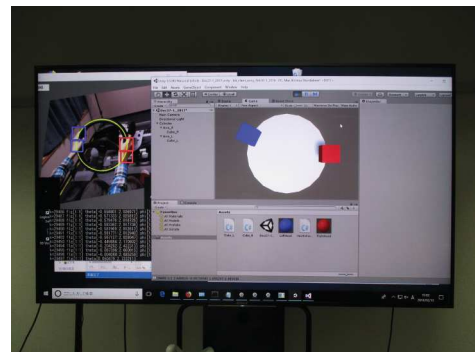
(a) Driver and Display : scene 1.



(b) Driver and Display : scene 2.



(c) Display : scene 3.

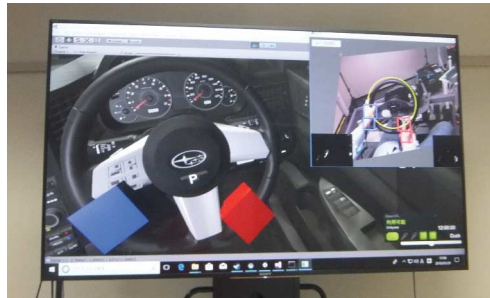


(d) Display : scene 4.

Fig. 4. Demonstration of Proposed System : Cylinder and Cube.



(a) Driver, Operator and VR Display : scene 5.



(b) VR Display : scene 6.

Fig. 5. Demonstration of Proposed System : Virtual Reality.

For the sake of simplicity, a cylinder model has been employed to represent steering, moreover, a blue cube and a red cube have been employed to represent left hand and right hand respectively. In Fig.4, left side of the display shows hands motion estimation result by the "Sensor Client for Hands Motion" with blue and red squares, while right side of the display shows CG graphics consisting of the cylinder and the cubes of blue/red colors drawn by the "Virtual Reality Client".

Second step of the implementation is to show the hands motion via head mounted display for car driver in the driving simulator facility. As a first trial for showing that, the cubes of the previous implementation in Unity system have also been employed as VR models of the hands as shown in Fig.5. In the figure, CG graphics displayed over the head mounted display have been shown at left side of the display with blue/red cubes drawn over the steering by the "Virtual Reality Client", while right side of the display shows hands motion estimation result by the "Sensor Client for Hands Motion" with blue and red squares, which is the same as the first step of the implementation explained above.

VI. DISCUSSION

Use of a simple camera connected via USB interface, of course, has limitations by relying only on the visual cues, typically, such as color of skin of hands/arms parts. Addition to this, in the driving simulator facility shown in Fig.3, background sight of the steering image captured by the camera varies due to two-axes motion of the cabin. This leads to difficulty for applying simply image processing under assumption of static background sight.



(a) Depth Sensor "Kinect" above Head of Driver.



(b) Depth Sensor "DepthSense" (DS325).

Fig. 6. Use of Depth Sensors.

To overcome or to circumvent the difficulty mentioned above, we discuss here some possibilities of use other sensors than simple camera, especially, the use of depth sensor such as Kinect, and the use of hand gesture sensor such as Leap Motion, in the following.

A. Depth Sensor

Use of depth image sensor such as Microsoft xbox 360 "Kinect" for hands motion estimation has been employed in [4] [5]. Depth information obtained by the depth sensor improves performance of correct pixel detection of hands/arms region over the image. Especially, arms region wearing sleeves can be detected by not only using skin color for detection but using nearer information to the sensor from the steering.

In addition with the use of depth sensors, high performance computing with GPGPU technology [6] [7] accelerates computational performance even by 10-100 times more particles for state estimation with particle filter leading to higher accuracy and faster computation.

Set-up position of the "Kinect" depth sensor is above head of a car driver as shown in Fig.6 (a), which is less realistic for actual use in a car due to large size of the sensor at the ceiling of car cabin especially for small size cars with short height to the ceiling.

Smaller depth sensors than "Kinect", such as "DepthSense" DS325/DS525 provided by SoftKinetic corporation (now it is acquired by SONY), as shown in Fig.6 (b), are more attractive due to its smaller size about 10[cm] width, which is comparative with typical high quality USB cameras sized about 8-9[cm] width. So it can be attached at the same position of the USB camera, as well as the position of "Kinect", due to its smaller size.

B. Hand Gesture Sensor

Latest technology provides more sophisticated hand gesture sensor "Leap Motion" provided by Leap Motion Inc., that is based on infrared light emission and infrared camera of fish-eye view with post-processing by software to grasp skeletons of hand/arm over the image. Resulted skeletons can be viewed

in overlaid manner on the captured infrared image (in gray scale), as well as in a virtual reality space by using Unity technology. Its small size about 8[cm] width in thin body is also attractive. Greatest advantage is the provided rich information of hand/arm posture in large degree of freedom consisting of length and direction of forearm, palm, and each part of all fingers. Note that the sensor is designed to use put on a desk, typically vertical direction to the flat desktop, during the use of a PC around the desk.

We have investigated utility of the hand gesture sensor "Leap Motion" for the purpose of hands motion estimation of car driver in operation of a car steering as follows. Set-up position of the sensor has been argued at first step of the investigation. As a result, one promising position has been determined as the same position as the USB camera set-up, i.e., above the seatbelt roller, as shown in Fig.7 (a), in which it is stacked over the USB camera in a context of possibly simultaneous use of both sensors.

Far distance of the sensor from the steering in this sensor set-up results to small aperture of hand/arm within captured image leading to larger detection error and tracking error. Especially, error of initial detection of hand/arm is significant, it would take seconds typically with intentional motion of hand/arm to be detected by the "Leap Motion" sensor. Thus, it is concluded that the simple use of "Leap Motion" is difficult for this purpose in this sensor set-up. Reason of this is misuse of the sensor, not on a desk near to the hands/arms, but attached at the place far from the hands/arms moving region in this scenario.

Other possibility to utilized the "Leap Motion" sensor for this purpose is to set it at front of the head mounted display as shown in Fig.7 (b), where special attachment has been designed and fabricated for the Oculus head mounted display. Note that, recently, Oculus provides a product with the attachment. In this set-up, hand/arm motion can be detected by the sensor correctly and can be used for the purpose of hands motion estimation of a car driver.

However, since the sensor FOV(Field-Of-View) depends on the head direction in this set-up, it becomes necessary to detect steering position over the space, in which hands/arms are in motion, in order to know grasping angular positions of hands over the steering. Furthermore, hands motion cannot be recorded during the head (i.e., FOV) directed to the other than the steering. This is OK for VR usage only, however, it is significant defect to estimate hands motion of a driver in steering operation always in a context of safety driving support or handover between human-driving and self-driving.

VII. CONCLUDING REMARKS

Proposal of this paper is one practical implementation method of a system consisting of 1) capturing hands motion in steering operation of car driver from vision sensor with the aid of state estimation by particle filter, 2) blackboard server system using TCP/IP socket communication in a framework of client/server in order to manage passing data among multiple computer as clients, and 3) displaying the driver's hands over

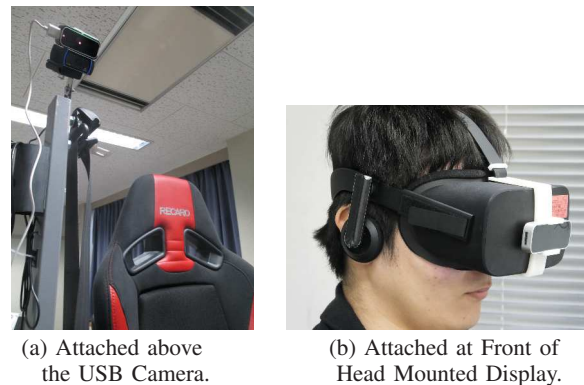


Fig. 7. Use of Hand Gesture Sensor "Leap Motion".

virtual reality graphics governed by Unity technology. Actual implementation result has been reported with several photos under driving simulator facility that equips two-axes motion actuators, HTC Vive head mounted display for virtual reality based on Unity technology, and driving simulator software engine. Discussions on utility of other sensors, depth sensors of "Kinect" / "DepthSense and hand gesture sensor "Leap Motion", have been made.

Future works include accomplishment of the implementation of virtual reality part for driving simulation in which the proposed system is necessary to work in real time. It will incorporate arms' angle information in the VR graphics in order to display the arms of car driver. Evaluation of the proposed system in terms of usability assessed by human subjects is also important issue as future work. In addition, possible applications based on the proposed method can be considered utilizing environmental sensors and virtual reality.

Acknowledgment This study was carried out under the ISM Cooperative Research Program (2018-ISM-CRP-2014).

REFERENCES

- [1] N.Ikoma, Y.Tanaka, Y.Inokuchi, I.Miyasita, K.Kawamoto, and T.Nishida, "Car Driver's Body Motion Estimation for Safety Driving using Particle Filter", Proc. of 4th Int'l Sympo. on Computational Intelligence and Industrial Applications (ISCIIA 2010), pp.17-25, 2010.
- [2] Norikazu Ikoma, "Visual tracking of both hands of car driver by particle filter", 5th Int'l Conf. on Soft Computing and Intelligent Systems and 11th Int'l Sympo. on advanced Intelligent Systems (SCIS & ISIS 2010), pp.1547-1552, 2010.
- [3] Norikazu Ikoma, "Real-Time Motion Estimation of Car Driver's Hands and Arm's Direction in Vision under Possible Mutual Occlusion by Particle Filter", 6th Int'l Conf. on Soft Computing and Intelligent Systems and 13th Int'l Sympo. on advanced Intelligent Systems (SCIS & ISIS 2012), pp.701-704, 2012.
- [4] Norikazu Ikoma, "Tracking of Car Driver's Hands in Depth Image Sensor by Particle Filter", 8th Int'l Conf. on Innovative Computing, Information and Control (ICICIC2013), A5-06, ICICIC2013-840, 2013.
- [5] Norikazu Ikoma, "Hands and arms motion estimation of a car driver with depth image sensor by using particle filter", The 14th Int'l Sympo. on Advanced Intelligent Systems (ISIS2013), 2013.
- [6] Norikazu Ikoma, "GPGPU Implementation for Steering Hands Tracking of a Car Driver by Particle Filter with Depth Image Sensor", 2nd Int'l Conf. on Robot, Vision and Signal Proc. (RVSP-2013), pp.164-167, 2013.
- [7] Norikazu Ikoma, "On GPGPU Parallel Implementation of Hands and Arms Motion Estimation of a Car Driver with Depth Image Sensor by Particle Filter", World Automation Congress 2014, 2014.